VTU B.E. PROJECT · SKIT · 2025–26

TEAM 14

Decentralized Proof of Personhood

On-device AI liveness detection + a non-transferable blockchain credential, proving "one wallet, one human" without a central identity authority.

Revanth • Shashank • Ajay • Chinmayee

Project overview prepared for guide review

1 Why We're Building This

> Sybil attacks — one person creating thousands of fake accounts or wallets — quietly break every system that assumes "one identity, one vote/entry/airdrop."

Bots and fake accounts are able to farm airdrops, dominate voting in DAOs, and drown out real users on social platforms, because there is no cheap, reliable way for a system to check "is this a unique real human?" without either:

- **Trusting a CAPTCHA** — increasingly defeated by automated solvers and bot farms.
- **Requiring government ID / KYC** — effective, but hands a centralized company or authority a permanent, trackable record of who you are, and excludes anyone without formal documentation.

Decentralized applications in particular — DAOs, quadratic voting, fair token distributions, community grants — need a **privacy-respecting** way to say "one human, one slot" that doesn't route through a central gatekeeper who can track, censor, or leak that data.

2 What We're Building

A decentralized Proof-of-Personhood system where a user proves they are a unique, live human being — using nothing but their webcam and a crypto wallet — and receives a **Soulbound Token (SBT)**: a non-transferable, on-chain credential that says "this wallet belongs to a verified human," without ever storing a name, ID document, or biometric template anywhere.

- **No ID documents.** Verification is behavioral (liveness), not identity-based.
- **No centralized database.** The only persistent record is a token on a public blockchain.
- **Non-transferable by design.** The credential cannot be sold, gifted, or moved — it stays bound to the wallet that earned it.

3 How It Works

1 User connects their MetaMask wallet (Polygon Amoy testnet).

2 App fetches a random liveness challenge — e.g. "*blink twice slowly.*"

3 On-device face tracking (MediaPipe) confirms the challenge was performed live, via webcam.

4 On success, verification metadata is pinned to IPFS.

5 A Soulbound Token (EIP-5192) is minted directly to the user's wallet.

6 Token is permanently locked to that wallet — one credential per human.

4 Architecture & Tech Stack

LAYER	TECHNOLOGY	ROLE
Smart Contract	Solidity, Hardhat, OpenZeppelin (ERC-721), EIP-5192	Mints and permanently locks the Soulbound Token; enforces one-per-wallet on-chain.
Backend	Python, FastAPI, Web3.py, Pinata (IPFS)	Issues challenges, pins verification metadata to IPFS, signs and submits the mint transaction.
Frontend	React, Vite, Tailwind CSS, ethers.js, MediaPipe Tasks Vision	Wallet connection, webcam capture, real-time on-device face-landmark liveness detection.
Network	Polygon Amoy Testnet	Low-cost, public chain to host the verification credential.

5 Real Engineering Challenges & How We Solved Them

These are concrete design problems we hit while building the system — not hypothetical risks — and the fixes actually implemented in the current codebase.

Liveness detection was slow, costly, and opaque with a cloud AI model

PROBLEM Our original design sent a recorded video clip to a cloud LLM (Gemini) for every single verification. That meant per-user upload latency, a running API cost for every verification, and — worst of all — an opaque "yes/no" judgment call from a model we couldn't fully audit for a security-relevant decision.

FIX We moved liveness detection entirely on-device using `MediaPipe FaceLandmarker` running in-browser (WASM/GPU). It tracks real-time face blendshapes (eye blink, smile, jaw, eyebrows) and head-pose angles, driving a deterministic state machine per challenge type. Result: verification is instant, free per-user, fully private (no video ever leaves the device), and auditable — the pass/fail logic is code we can read, not a model prompt.

Stopping one wallet from minting multiple credentials

PROBLEM Nothing stopped a user from calling verify twice and getting two Soulbound Tokens minted to the same wallet — breaking the entire "one human, one credential" premise, and wasting gas.

FIX Enforced at two layers: the backend checks the contract's `isVerified` mapping before doing any work, and — critically — the contract itself rejects a second mint with `require(!isVerified[to])`. Even if the backend check is bypassed or raced, the chain is the final source of truth.

A standard NFT can be sold — that defeats the whole point

PROBLEM An ordinary ERC-721 token can be transferred after mint. If a verified human could sell their "proof of personhood" to someone else, the credential would stop meaning anything.

FIX We override OpenZeppelin's internal transfer hook so any transfer is rejected unless it's the initial mint (`from == address(0)`), and implement the EIP-5192 `locked()` standard so wallets and explorers correctly display it as a non-transferable "soulbound" token rather than a broken one.

Browser blocked frontend → backend requests during local development

PROBLEM The React dev server (port 5173) and the FastAPI backend (port 8000) are different origins, so the browser's default same-origin policy silently blocked every API call during early testing.

FIX Added explicit CORS middleware in FastAPI, allow-listing only our known local dev ports rather than opening it to every origin.

Teammates without funded wallets or API keys couldn't run the app

PROBLEM Testing the full flow required a live Pinata account and a gas-funded Polygon Amoy wallet with a deployed contract. Any teammate missing those — or hitting an empty testnet faucet — was blocked from working on anything else.

FIX The backend degrades gracefully: it returns a mock IPFS URI when Pinata keys aren't set, and skips the on-chain mint step entirely when the contract address or minter key are missing or still placeholder values — so the liveness/UI flow can be built and tested independently of blockchain access.

Hardhat's Node.js compatibility warning

PROBLEM Our dev environment runs Node.js v18.16.1, which Hardhat flags as unsupported — it can cause unpredictable compiler/plugin behavior that's hard to reproduce.

FIX Currently tracked as an open item — we compile and test successfully today, but plan to upgrade to a Hardhat-supported Node LTS release before final deployment to rule out subtle version-related issues.

6 Current Status

- **Smart contract** — written, compiles cleanly, deploy script ready for Polygon Amoy.
- **Frontend** — builds and runs; wallet connect, webcam capture, and live challenge UI complete.
- **Liveness detection** — on-device MediaPipe pipeline working for 7 challenge types.
- **Backend** — FastAPI service serving challenges, IPFS pinning, and mint logic complete.
- **End-to-end on testnet** — pending funded wallet + live Pinata keys for full demo run.
- **Node/Hardhat version** — works today; LTS upgrade planned before final submission.

7 Team 14

Revanth • Shashank • Ajay • Chinmayee

VTU B.E. Final Year Project — SKIT, 2025–26